



5.6 switch Multiple-Selection Statement

- ▶ The **switch multiple-selection** statement performs many different actions based on the possible values of a variable or expression.
- ▶ Each action is associated with the value of an **integral constant expression** (i.e., any combination of character and integer constants that evaluates to a constant integer value).

5.6 switch Multiple-Selection Statement (cont.)



switch Statement Details

- ▶ The `switch` statement consists of a series of `case labels` and an optional `default case`.
- ▶ When the flow of control reaches the `switch`, the program evaluates the expression in the parentheses.
 - The `controlling expression`.
- ▶ The `switch` statement compares the value of the controlling expression with each `case` label.
- ▶ If a match occurs, the program executes the statements for that `case`.
- ▶ The `break` statement causes program control to proceed with the first statement after the `switch`.

5.6 switch Multiple-Selection Statement (cont.)



- ▶ Each **case** can have multiple statements.
 - The **switch** selection statement does not require braces around multiple statements in each **case**.
- ▶ Without **break** statements, each time a match occurs in the **switch**, the statements for that **case** *and* subsequent **cases** execute until a **break** statement or the end of the **switch** is encountered.
 - Referred to as “falling through” to the statements in subsequent **cases**.



Common Programming Error 5.6

Forgetting a **break** statement when one is needed in a **switch** statement is a logic error.



Common Programming Error 5.7

Omitting the space between the word `case` and the integral value tested in a `switch` statement—e.g., writing `case3:` instead of `case 3:`—is a logic error. The `switch` statement will not perform the appropriate actions when the controlling expression has a value of 3.

5.6 switch Multiple-Selection Statement (cont.)



Providing a default Case

- ▶ If *no* match occurs between the controlling expression's value and a **case** label, the **default** case executes.
- ▶ If no match occurs in a **switch** statement that does not contain a **default** case, program control continues with the first statement after the **switch**.

5.6 switch Multiple-Selection Statement (cont.)



```
switch (expression)
{
  case constant1:
    group of statements 1;
    break;
  case constant2:
    group of statements 2;
    break;
  .
  .
  .
  default:
    default group of statements
}
```



Good Programming Practice 5.4

The last `case` in a `switch` statement does not require a `break` statement. Nevertheless, include this `break` for clarity and for symmetry with other cases.

5.6 switch Multiple-Selection Statement (cont.)



switch Statement UML Activity Diagram

- ▶ Figure 5.12 shows the UML activity diagram for the general **switch** multiple-selection statement.
- ▶ Most **switch** statements use a **break** in each **case** to terminate the **switch** statement after processing the **case**.
- ▶ Figure 5.12 emphasizes this by including **break** statements in the activity diagram.
- ▶ Without the **break** statement, control would not transfer to the first statement after the **switch** statement after a **case** is processed.
- ▶ Instead, control would transfer to the next **case**'s actions.
- ▶ The diagram makes it clear that the **break** statement at the end of a **case** causes control to exit the **switch** statement immediately.

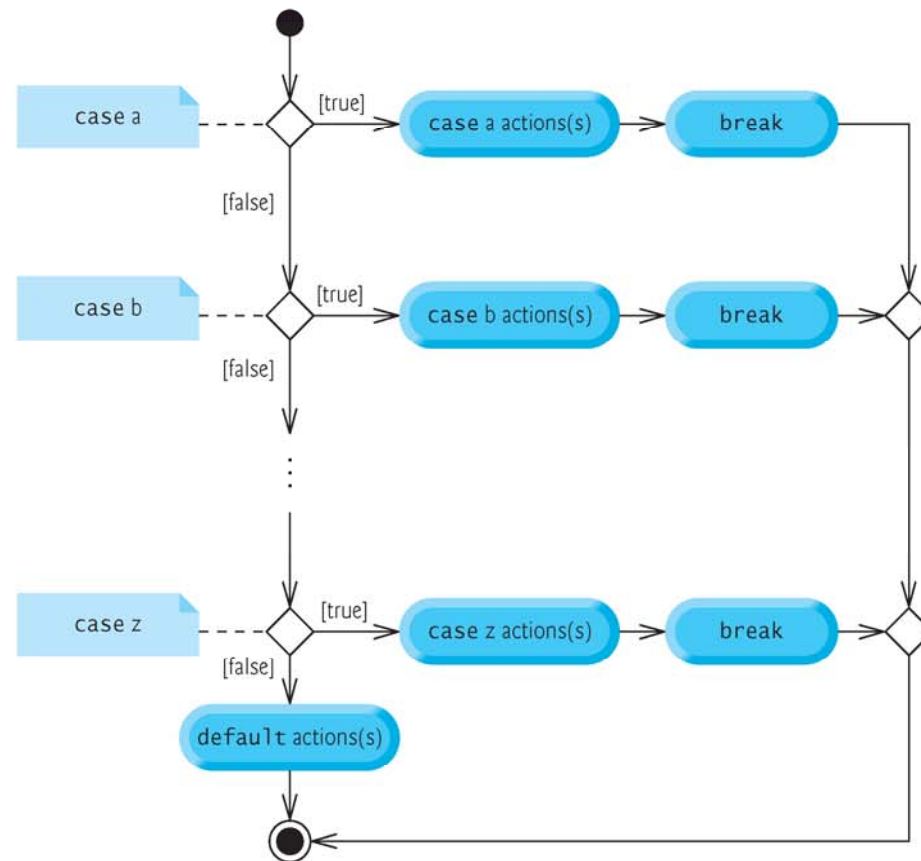


Fig. 5.12 | switch multiple-selection statement UML activity diagram with break statements.



Common Programming Error 5.9

Providing `case` labels with identical values in a `switch` statement is a compilation error.



Both of the following code fragments have the same behavior:

switch example	if-else equivalent
<pre>switch (x) { case 1: cout << "x is 1"; break; case 2: cout << "x is 2"; break; default: cout << "value of x unknown"; }</pre>	<pre>if (x == 1) { cout << "x is 1"; } else if (x == 2) { cout << "x is 2"; } else { cout << "value of x unknown"; }</pre>

Example:–



```
#include <iostream>
using namespace std;

1. void main()
2. {
3.     unsigned short int number;
4.     cout << "Enter a number between 1 and 5: ";
5.     cin >> number;
6.     switch (number)
7.     {
8.         case 0: cout << "Too small, sorry!";
9.         break;
10.        case 5: cout << "Good job! " << endl; // fall through
11.        case 4: cout << "Nice Pick!" << endl; // fall through
12.        case 3: cout << "Excellent!" << endl; // fall through
13.        case 2: cout << "Masterful!" << endl; // fall through
14.        case 1: cout << "Incredible!" << endl;
15.        break;
16.        default: cout << "Too large!" << endl;
17.        break;
18.    }
19.    cout << endl << endl;
20.    }
```



Notice that switch can only be used to compare an expression against constants. Therefore we cannot put variables as labels (for example case n: where n is a variable) or ranges (case (1..3):) because they are not valid C++ constants.

If you need to check ranges or values that are not constants, use a concatenation of if and else if statements.

o/p of the program:

Enter a number between 1 and 5: 3

Excellent!

Masterful!

Incredible!

Enter a number between 1 and 5: 8

Too large!



analysis:

- ▶ The user is prompted for a number on lines 5. That number is given to the switch statement on line 6. If the number is 0, the case statement on line 8 matches, the message **Too small, sorry!** is printed, and the break statement on line 9 ends the switch. If the value is 5, execution switches to line 10 where a message is printed, and then falls through to line 11, another message is printed, and so forth until hitting the break on line 15, at which time the switch ends.
- ▶ The net effect of these statements is that for a number between 1 and 5, that many messages are printed. If the value of number is not 0 to 5, it is assumed to be too large, and the default statement is invoked on line 16.



5.7 break and continue Statements

break Statement

- ▶ The **break statement**, when executed in a **while**, **for**, **do...while** or **switch** statement, causes immediate exit from that statement.
- ▶ Program execution continues with the next statement.
- ▶ Common uses of the **break** statement are to escape early from a loop or to skip the remainder of a **switch** statement.
- ▶ Figure 5.13 demonstrates the **break** statement (line 13) exiting a **for** repetition statement.



```
1 // Fig. 5.13: fig05_13.cpp
2 // break statement exiting a for statement.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     unsigned int count; // control variable also used after loop terminates
9
10    for ( count = 1; count <= 10; ++count ) // loop 10 times
11    {
12        if ( count == 5 )
13            break; // break loop only if count is 5
14
15        cout << count << " ";
16    } // end for
17
18    cout << "\nBroke out of loop at count = " << count << endl;
19 }
```

```
1 2 3 4
Broke out of loop at count = 5
```

Fig. 5.13 | break statement exiting a for statement.



5.7 break and continue Statements (cont.)

- ▶ The **continue statement**, when executed in a **while**, **for** or **do...while** statement, skips the remaining statements in the body of that statement and proceeds with the next iteration of the loop.
- ▶ In **while** and **do...while** statements, the loop-continuation test evaluates immediately after the **continue** statement executes.
- ▶ In the **for** statement, the increment expression executes, then the loop-continuation test evaluates.



```
1 // Fig. 5.14: fig05_14.cpp
2 // continue statement terminating an iteration of a for statement.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     for ( unsigned int count = 1; count <= 10; ++count ) // loop 10 times
9     {
10         if ( count == 5 ) // if count is 5,
11             continue;    // skip remaining code in loop
12
13         cout << count << " ";
14     } // end for
15
16     cout << "\nUsed continue to skip printing 5" << endl;
17 } // end main
```

```
1 2 3 4 6 7 8 9 10
Used continue to skip printing 5
```

Fig. 5.14 | continue statement terminating an iteration of a for statement.



5.8 Logical Operators

- ▶ C++ provides **logical operators** that are used to form more complex conditions by combining simple conditions.
- ▶ The logical operators are `&&` (logical AND), `||` (logical OR) and `!` (logical NOT, also called logical negation).



5.8 Logical Operators (cont.)

Logical AND (&&) Operator

- ▶ The **&&** (logical AND) operator is used to ensure that two conditions are *both true* before we choose a certain path of execution.
- ▶ The simple condition to the left of the **&&** operator evaluates first.
- ▶ If necessary, the simple condition to the right of the **&&** operator evaluates next.
- ▶ The right side of a logical AND expression is evaluated only if the left side is **true**.



Common Programming Error 5.10

Although $3 < x < 7$ is a mathematically correct condition, it does not evaluate as you might expect in C++. Use `( 3 < x && x < 7 )` to get the proper evaluation in C++.



5.10 Logical Operators (cont.)

- ▶ Figure 5.15 summarizes the `&&` operator.
- ▶ The table shows all four possible combinations of `false` and `true` values for *expression1* and *expression2*.
- ▶ Such tables are often called **truth tables**.
- ▶ C++ evaluates to `false` or `true` all expressions that include relational operators, equality operators and/or logical operators.



expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Fig. 5.15 | && (logical AND) operator truth table.



5.8 Logical Operators (cont.)

Logical OR (| |) Operator

- ▶ The `||` (logical OR) operator determines if either *or* both of two conditions are `true` before we choose a certain path of execution.
- ▶ Figure 5.16 is a truth table for the logical OR operator (`||`).
- ▶ The `&&` operator has a higher precedence than the `||` operator.
- ▶ Both operators associate from left to right.
- ▶ An expression containing `&&` or `||` operators evaluates only until the truth or falsehood of the expression is known.
 - This performance feature for the evaluation of logical AND and logical OR expressions is called **short-circuit evaluation**.



expression1	expression2	expression1 expression2
false	false	false
false	true	true
true	false	true
true	true	true

Fig. 5.16 | || (logical OR) operator truth table.



5.8 Logical Operators (cont.)

Logical Negation (!) Operator

- ▶ C++ provides the **!** (**logical NOT**, also called **logical negation**) operator to “reverse” a condition’s meaning.
- ▶ The unary logical negation operator has only a single condition as an operand.
- ▶ You can often avoid the **!** operator by using an appropriate relational or equality operator.
- ▶ Figure 5.17 is a truth table for the logical negation operator (**!**).



expression	! expression
false	true
true	false

Fig. 5.17 | ! (logical negation)
operator truth table.

```
!(5 == 5)    // evaluates to false because the expression at its right (5 == 5) is true.  
!(6 <= 4)    // evaluates to true because (6 <= 4) would be false.
```



5.9 Confusing the Equality (==) and Assignment (=) Operators

- ▶ Accidentally swapping the operators == (equality) and = (assignment).
- ▶ Damaging because they ordinarily do not cause syntax errors.
- ▶ Rather, statements with these errors tend to compile correctly and the programs run to completion, often generating incorrect results through runtime logic errors.
- ▶ [*Note: Some compilers issue a warning when = is used in a context where == typically is expected.*]
- ▶ Two aspects of C++ contribute to these problems.
 - One is that *any expression that produces a value can be used in the decision portion of any control statement.*
 - The second is that assignments produce a value—namely, the value assigned to the variable on the left side of the assignment operator.
- ▶ *Any nonzero value is interpreted as true.*



Error-Prevention Tip 5.4

Programmers normally write conditions such as `x == 7` with the variable name on the left and the constant on the right. By placing the constant on the left, as in `7 == x`, you'll be protected by the compiler if you accidentally replace the `==` operator with `=`. The compiler treats this as a compilation error, because you can't change the value of a constant. This will prevent the potential devastation of a runtime logic error.